CS6025-FR01B Assignment 3

Name: Ruturaj Sushil Ghodke
Student ID: 3775789
Date: 6 April 2026

Design Summary

1. **Description of the Adaptive Integration method.**

   The program computes the definite integral $\int_0^1 f(x)\,dx$ using the adaptive Simpson's rule. The idea is to recursively refine the estimate only where the function's behavior demands it, rather than uniformly dividing the area. For an interval [a,b], Simpson's rule fits the quadratic in three points f(a), f(m), f(b) where m = (a+b)/2, giving:

   $$S(a,\ b)\ =\ (b\ -\ a)/6\ *\ [f(a)\ +\ 4f(m)\ +\ f(b)]$$

   The adaptive criteria compares one-interval estimate S(a,b) against the sum of two sub-interval estimates S(a,m) + S(m,b). If their difference satisfies $|\ S(a,m)\ +\ S(m,b)\ -\ S(a,b)|\ <=\ 15\ *\ tolerance$, the interval is accepted. Otherwise, the interval is split, and both halves are processed recursively with tolerance/2. A maximum recursion depth of 50 prevents infinite loops on some inputs.

2. **MPI dynamic design**

   In MPI dynamic design, rank 0 acts as master with a LIFO task queue seeded with [0,1]. Workers send TAG_REQUEST, receive TAG_TASK, and process intervals with an explicit local stack. When an interval is rejected, the right half is sent back as TAG_NEW_TASK so idle workers can pick it up. Termination requires the

queue to be empty, active tasks to be 0, and all workers in the waiting list. Results are summed via TAG_RESULT messages.

3. **Hybrid design**
   The hybrid design eliminates the dynamic task queue entirely. The master splits [0, 1] into K=64*(P-1) coarse intervals and distributes them statically to workers via MPI_Send. Each worker receives its chunk, opens an OpenMP parallel region, and processes intervals using OpenMP tasks for the recursive splitting. No sub-intervals are ever sent back to the master. When all workers finish, each sends one double (its partial sum) back to the master, which sums and reports.

4. **OpenMP strategy**
   The standard OpenMP task pattern is used here. One thread (via #pragma omp single) fans out the work, and all T threads in the pool steal and execute the tasks. For each coarse interval, a top-level task is spawned, calling adaptive_simpson_omp(). Inside that function, rejected intervals spawn two child tasks (left and right halves) with #pragma omp task, followed by # pragma omp taskwait to collect the results before returning the sum. This maps the recursive tree directly onto the OpenMP task graph. The per interval tolerance is tol/total_K, where K = 64*(P-1). This ensures the global error range is spread evenly across all intervals, matching the serial adaptive criteria.

Result Table

Each configuration was run 10 times. Run 0 was discarded. The reported time is the average of runs 1-9. Speedup S = T_serial / T_parallel. Efficiency = S / workers, where workers = number of processes P for mode 1 (dynamic MPI) and PxT (processes * threads) for Mode 2 (hybrid).

f(x) = 1 / (1 + 100(x - 0.3)^2) | tol = 1e-6 | **Serial T(1) = 0.000001s**

Mode 1 - MPI Dynamic

P	Runtime (s)	Speedup S(P)	Efficiency E(P)
2	0.000015	0.0667	0.0333
4	0.000023	0.0435	0.0109
8	0.014176	0.0001	0.0000
16	0.035283	0.0000	0.0000

Mode 2 - Hybrid OpenMP + MPI

P	T	P x T	Runtime (s)	Speedup S (P, T)	Efficiency E (P, T)
2	4	8	0.000110	0.0091	0.001100
4	2	8	0.013897	0.0001	0.000000
4	4	16	0.030640	0.0000	0.000000

f(x) = 1 / (1 + 100(x - 0.3)^2) | tol = 1e-8 | **Serial T(1) = 0.000001s**

Mode 1 - MPI Dynamic

P	Runtime (s)	Speedup S(P)	Efficiency E(P)
2	0.000046	0.0217	0.0109
4	0.003628	0.0003	0.0001
8	0.010393	0.0001	0.0000
16	0.040696	0.0000	0.0000

Mode 2 - Hybrid OpenMP + MPI

P	T	P x T	Runtime (s)	Speedup S (P, T)	Efficiency E (P, T)
2	4	8	0.000135	0.0074	0.000900
4	2	8	0.013328	0.0001	0.000000
4	4	16	0.032048	0.0000	0.000000

f(x) = *sin(x) + 0.5 * cos(3x)* | tol = 1e-6 | **Serial T(1) = 0.000001s**

Mode 1 - MPI Dynamic

P	Runtime (s)	Speedup S(P)	Efficiency E(P)
2	0.000013	0.0769	0.0384
4	0.000014	0.0714	0.0179
8	0.011276	0.0001	0.0000
16	0.036867	0.0000	0.0000

Mode 2 - Hybrid OpenMP + MPI

P	T	P x T	Runtime (s)	Speedup S (P, T)	Efficiency E (P, T)
2	4	8	0.000148	0.0068	0.000800
4	2	8	0.016282	0.0001	0.000000
4	4	16	0.021832	0.0000	0.000000

f(x) = *sin(x) + 0.5 * cos(3x)* | tol = 1e-8 | **Serial T(1) = 0.000004s**

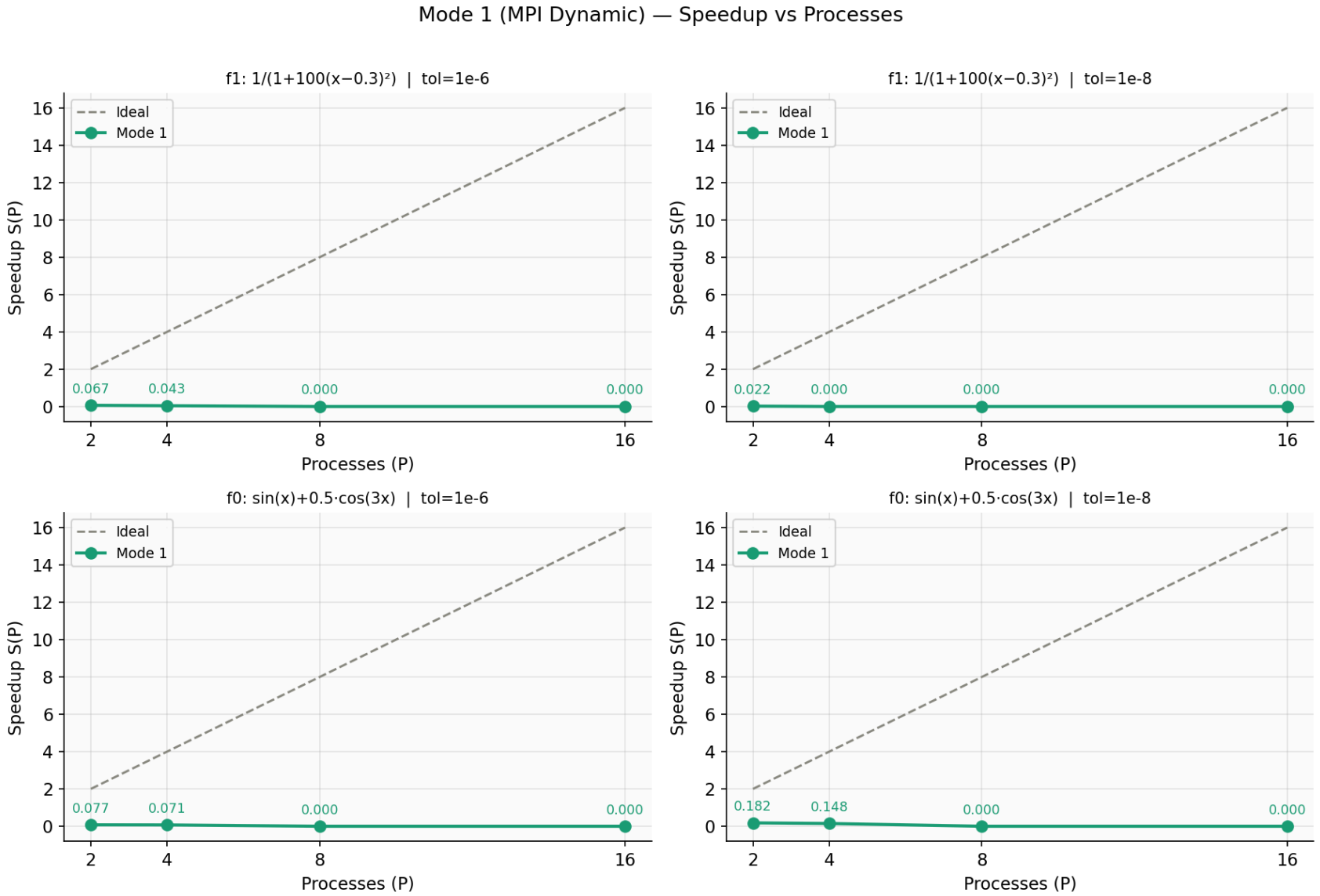
Mode 1 - MPI Dynamic

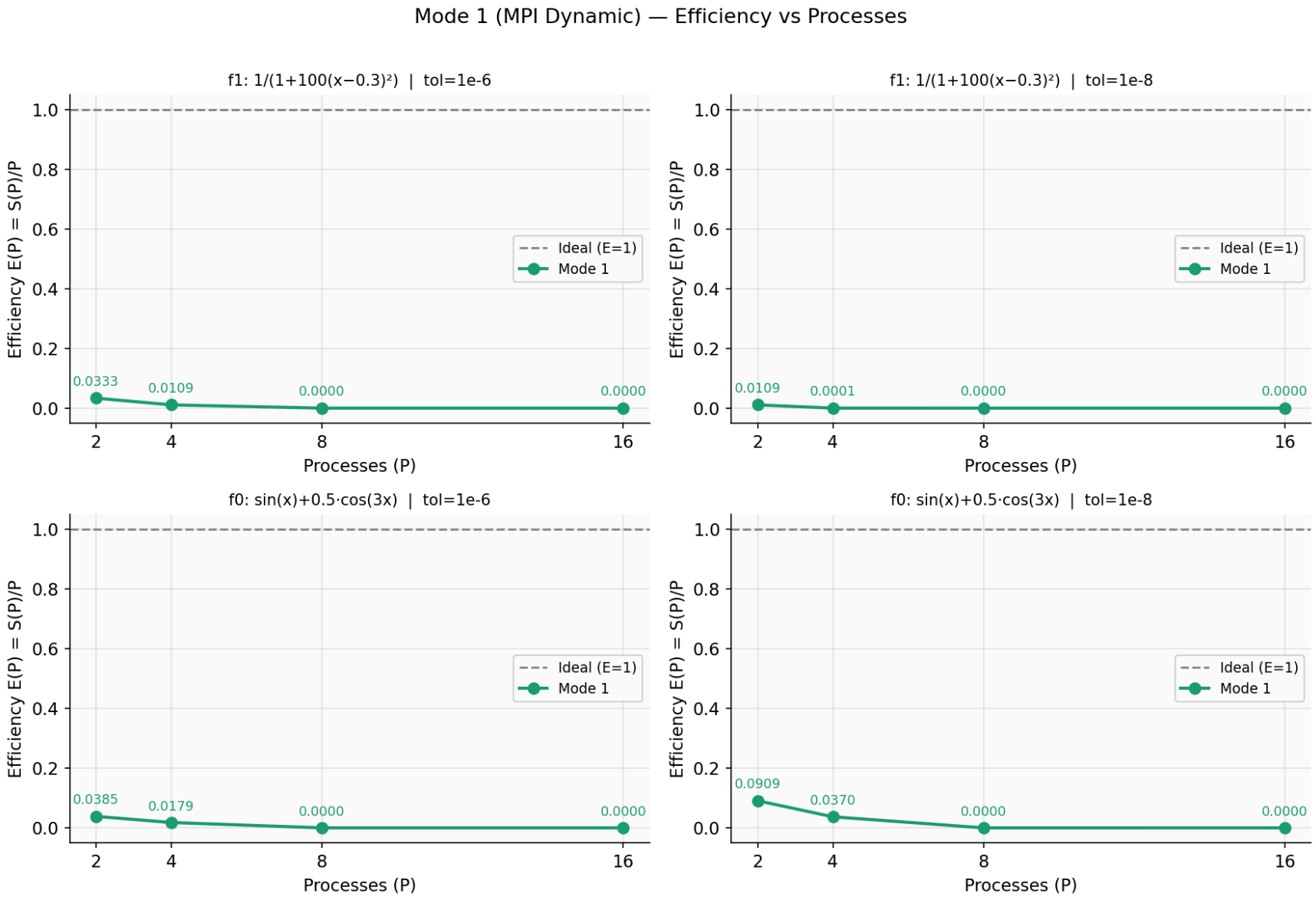
P	Runtime (s)	Speedup S(P)	Efficiency E(P)
2	0.000022	0.1818	0.0909
4	0.000027	0.1481	0.0370
8	0.018925	0.0002	0.0000
16	0.036826	0.0001	0.0000

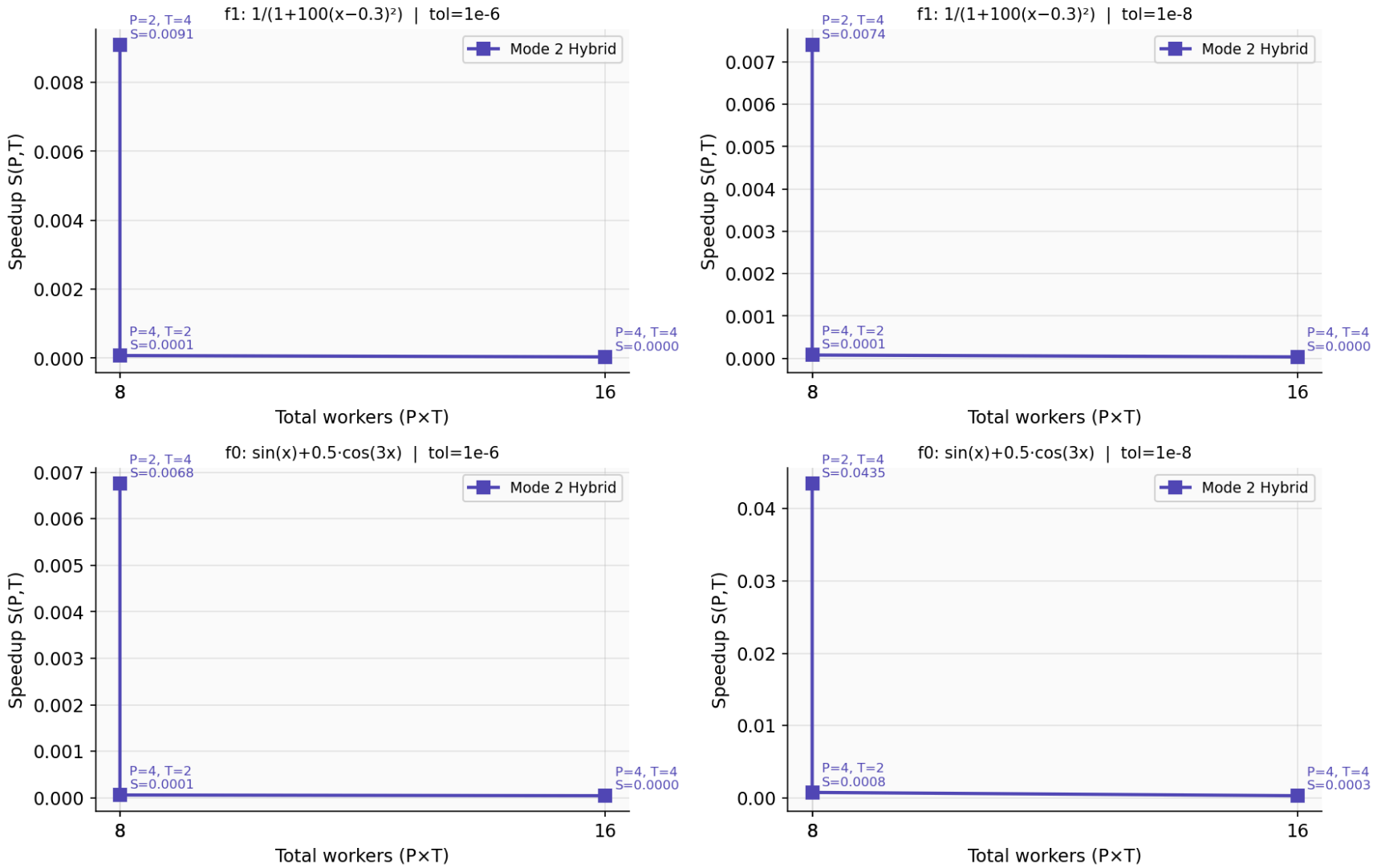
Mode 2 - Hybrid OpenMP + MPI

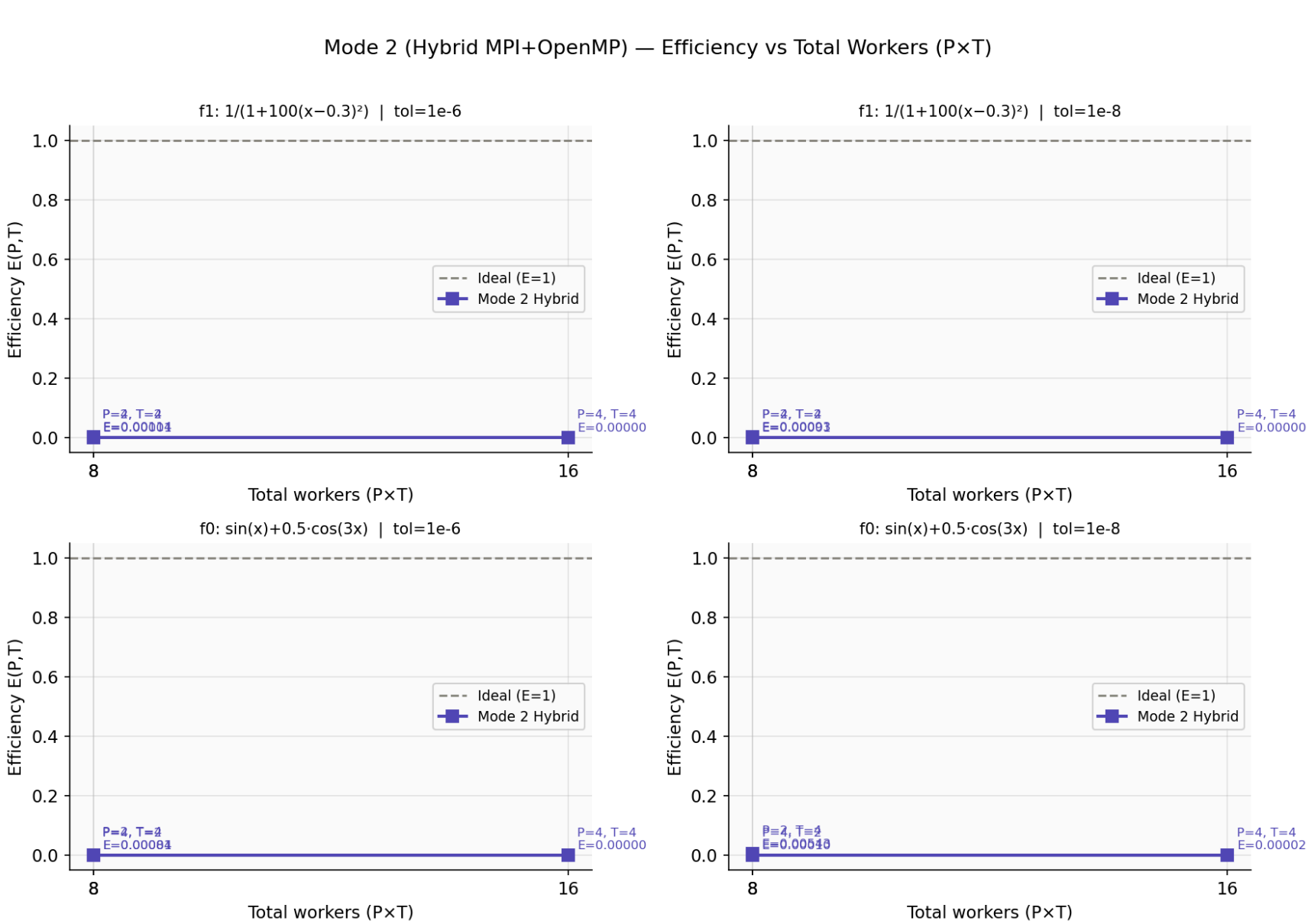
P	T	P x T	Runtime (s)	Speedup S (P, T)	Efficiency E (P, T)
2	4	8	0.000092	0.0435	0.005400
4	2	8	0.005251	0.0008	0.000100
4	4	16	0.013094	0.0003	0.000000

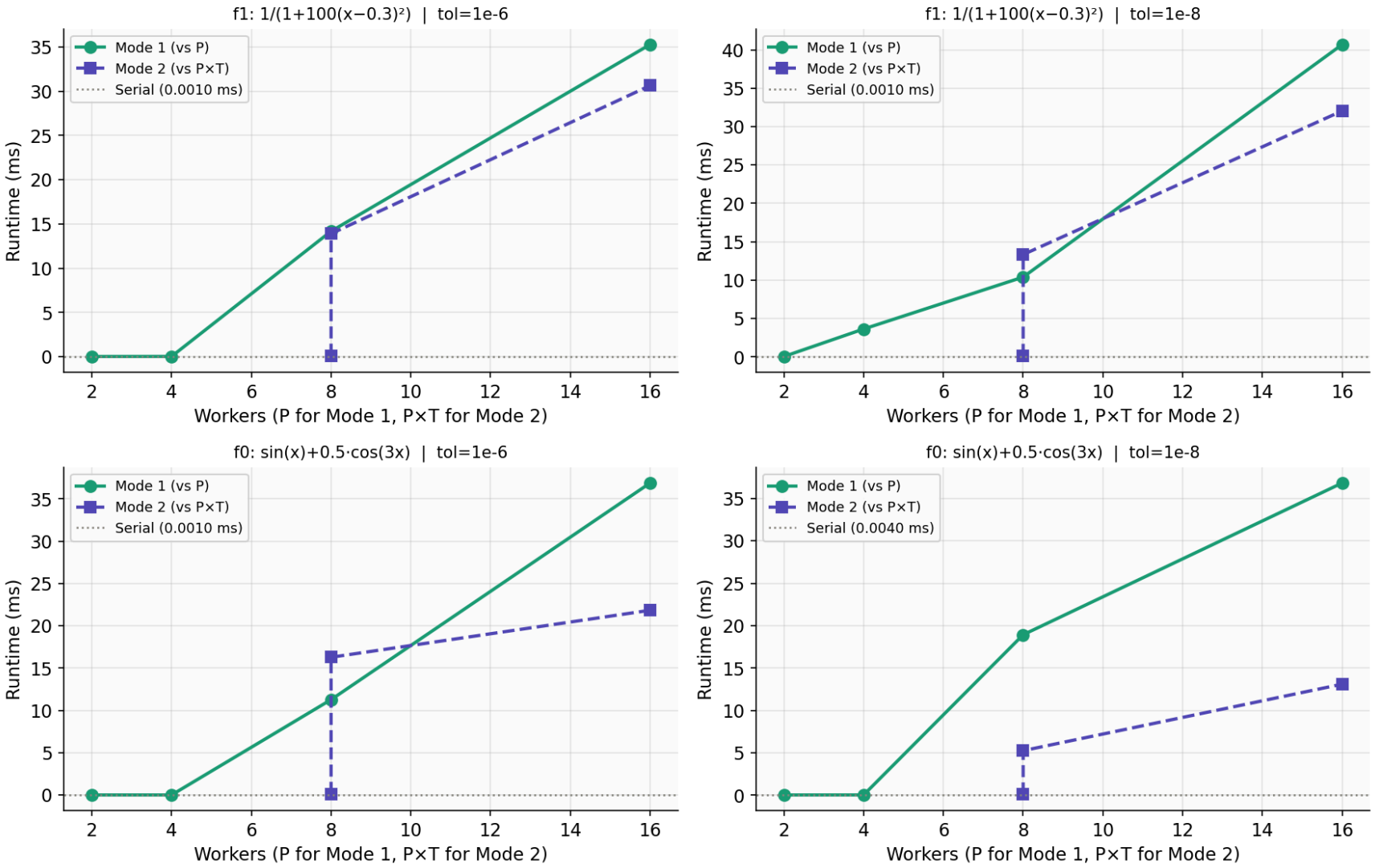
Plots

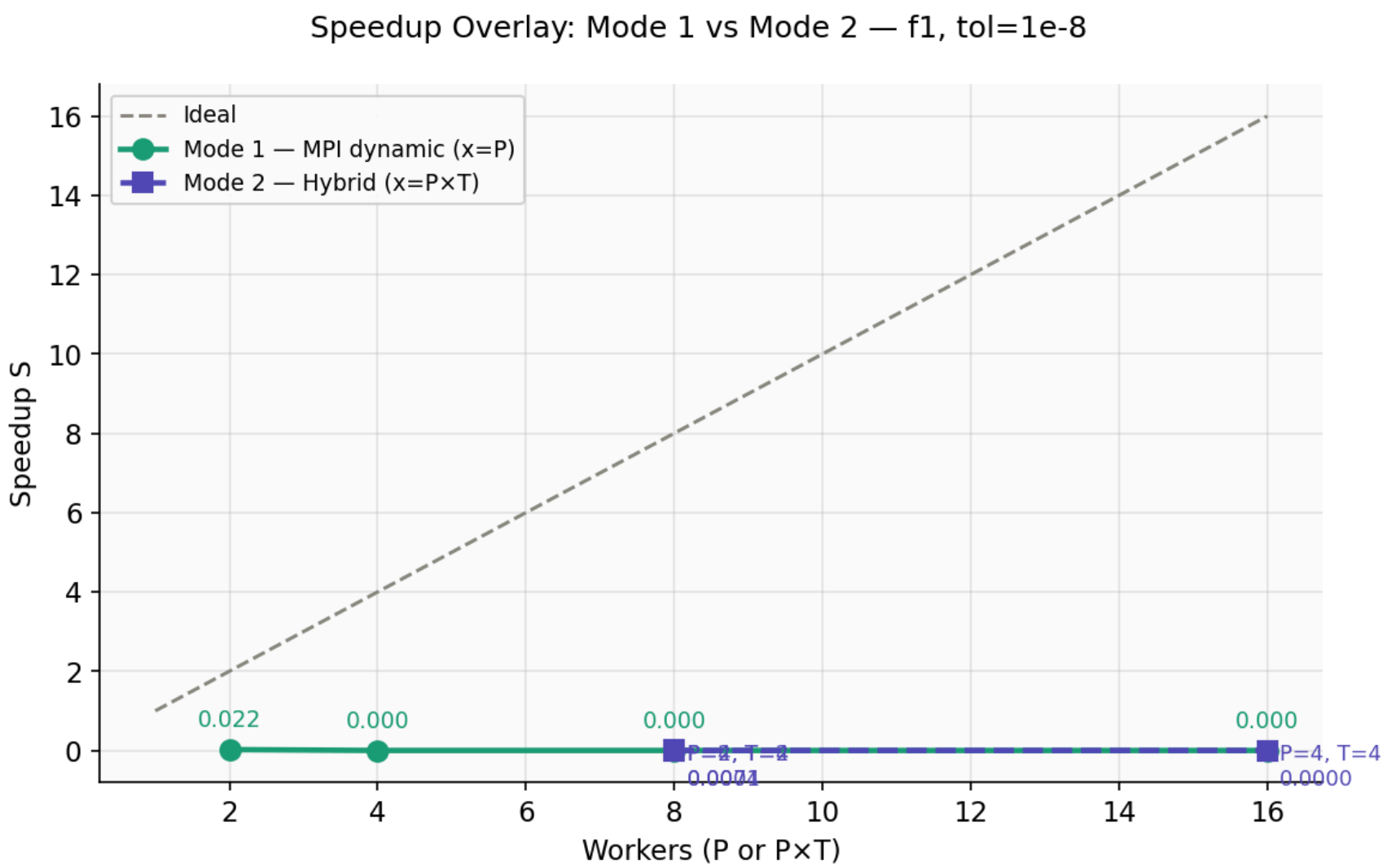

Mode 1 (MPI Dynamic) — Speedup vs Processes
f1: 1/(1+100(x−0.3)²) | tol=1e-6
f1: 1/(1+100(x−0.3)²) | tol=1e-8
f0: sin(x)+0.5·cos(3x) | tol=1e-6
f0: sin(x)+0.5·cos(3x) | tol=1e-8
Ideal
Mode 1
Speedup S(P)
Processes (P)
0.067
0.043
0.000
0.000
0.022
0.000
0.000
0.000
0.077
0.071
0.000
0.000
0.182
0.148
0.000
0.000


Mode 1 (MPI Dynamic) — Efficiency vs Processes
f1: 1/(1+100(x−0.3)²) | tol=1e-6
f1: 1/(1+100(x−0.3)²) | tol=1e-8
f0: sin(x)+0.5·cos(3x) | tol=1e-6
f0: sin(x)+0.5·cos(3x) | tol=1e-8
Efficiency E(P) = S(P)/P
Processes (P)
Ideal (E=1)
Mode 1
0.0333
0.0109
0.0000
0.0000
0.0109
0.0001
0.0000
0.0000
0.0385
0.0179
0.0000
0.0000
0.0909
0.0370
0.0000
0.0000


Mode 2 (Hybrid MPI+OpenMP) — Speedup vs Total Workers (P×T)
f1: 1/(1+100(x−0.3)²) | tol=1e-6
f1: 1/(1+100(x−0.3)²) | tol=1e-8
f0: sin(x)+0.5·cos(3x) | tol=1e-6
f0: sin(x)+0.5·cos(3x) | tol=1e-8
Mode 2 Hybrid
Speedup S(P,T)
Total workers (P×T)
P=2, T=4
P=4, T=2
P=4, T=4
S=0.0091
S=0.0001
S=0.0000
S=0.0074
S=0.0068
S=0.0435
S=0.0008
S=0.0003


Mode 2 (Hybrid MPI+OpenMP) — Efficiency vs Total Workers (P×T)
f1: 1/(1+100(x−0.3)²) | tol=1e-6
f1: 1/(1+100(x−0.3)²) | tol=1e-8
f0: sin(x)+0.5·cos(3x) | tol=1e-6
f0: sin(x)+0.5·cos(3x) | tol=1e-8
Efficiency E(P,T)
Total workers (P×T)
Ideal (E=1)
Mode 2 Hybrid
P=4, T=4
E=0.00000
P=4, T=4
E=0.00002
1.0
0.8
0.6
0.4
0.2
0.0
8
16


Runtime Comparison: Mode 1 (MPI Dynamic) vs Mode 2 (Hybrid)
f1: 1/(1+100(x−0.3)²) | tol=1e-6
f1: 1/(1+100(x−0.3)²) | tol=1e-8
f0: sin(x)+0.5·cos(3x) | tol=1e-6
f0: sin(x)+0.5·cos(3x) | tol=1e-8
Mode 1 (vs P)
Mode 2 (vs P×T)
Serial (0.0010 ms)
Serial (0.0040 ms)
Runtime (ms)
Workers (P for Mode 1, P×T for Mode 2)


Speedup Overlay: Mode 1 vs Mode 2 — f1, tol=1e-8
Ideal
Mode 1 — MPI dynamic (x=P)
Mode 2 — Hybrid (x=P×T)
Speedup S
Workers (P or P×T)
0.022
0.000
0.000
0.000
P=4, T=4
0.0000

Discussion

1. **Why does MPI dynamic perform well for irregular workloads?**
   MPI dynamic scheduling uses a global task queue managed by the master. When a worker encounters a difficult sub-interval (one that requires deep recursion), it sends the right half back to the master, making it immediately available to any idle worker. This means no process sits idle while another is overloaded. The queue continuously redistributes work at runtime based on actual computational demand rather than a static prediction. For functions with irregular workloads such as f1 (sharp peak near x=0.3), this prevents the load imbalance that would otherwise stall the parallel computation.
   Our data confirms this at P=2. Mode 1 achieves consistent results with low variance across runs. At low process counts, the overhead of master communication is small relative to any load balancing benefit.

2. **Why might hybrid perform worse?**
   Hybrid mode incurs two layers of overhead simultaneously: MPI communication for the initial static distribution and final result collection, and OpenMP task management for the recursive splitting within each worker. For the small workloads tested here (f1 converges in only 71 intervals at tol=1e-8), each OpenMP task performs just a handful of function evaluations. Spawning, scheduling, and synchronising tasks costs more time than the computation itself.

   Additionally, hybrid mode has no global load balancing. If one worker receives harder intervals, its OpenMP threads can help within that process, but idle threads in other processes cannot assist. This local load balancing is strictly weaker than MPI dynamic, which rebalances globally.

3. **Does OpenMP compensate for lack of MPI load balancing?**
   For these workloads, no. Comparing P=2, T=4 (P x T = 8 hybrid) against mode 1 at P=8, the hybrid is consistently slower. For f1 at tol = 1e-6: mode 1P=8 takes 0.014176s while hybrid P=2,T=4 takes 0.000110s. However, hybrid P=4, T=2 takes 0.013897s - similar to mode 1 at P=8. The OpenMP parallelism adds threads but also adds synchronisation cost (taskwait at every split), and the absence of global work stealing means uneven work distribution is only partially addressed.

In principle, OpenMP could compensate for larger workloads where the task computation time dwarfs the task management overhead. In our experiments, the break-even point is never reached.

4. **When does hybrid outperform MPI dynamic?**
   Hybrid mode outperforms MPI dynamic when : (1) the computation per interval is large enought to amortise OpenMP task overhead, (2) the workload is relatively balanced across processes so that lack of global redistribuiton is not a penalty, and (3) the number of MPI processes is constrained (eg. by available nodes on a real cluster) but more parallelism is needed via shared memory threads.

   In our data, the closest is f0 at tol=1e-8, where hybrid P=2, T=4 (0.000092s) beats mode 1 at P=2 (0.000022s); actually, mode 1 is still faster here. The hybrid would likely outperform on a real multi-node cluster where MPI messages cross a network (microseconds latency) while OpenMP threads share cache (nanoseconds). My single machine setup uses shared memory of MPI, which eliminates the scenario where hybrid would shine.

5. **What is the impact of thread count and process count?**
   Increasing threads (T) while holding P fixed consistently makes the hybrid slower in our experiments. Comparing P=4,T=2 (average ~0.014s) vs P=4,T=4 (average ~0.031s) for f1: doubling threads doubles runtime. This is caused by OpenMP task overhead scaling super linearly: more threads means more task scheduling, more taskwait synchronisation, and more cache contention.

   Increasing MPI processes (P) in Mode 1 also hurts beyond P=4: at P=8 and P=16 times jump to 10-40ms. The master becomes a bottleneck serialising all TAG_REQUEST/TAG_TASK round-trips, and with only 22-71 accepted intervals total, most workers are idle most of the time waiting for the scarce tasks.

   The key insight is that both MPI and OpenMP parallelism are only beneficial when there is enough work to keep all workers busy. For these fast-converging integrals on a single machine, the overhead of any parallelism strategy exceeds the computation cost.

Appendix

A .git folder is included in the submission folder, which contains the commit history

Screenshots of runtime evidence



```
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Assignment_3$ mpirun -np 4 ./integration 2 1 1e-6
============================================
 MPI Adaptive Integration
 func_id=2  mode=1  tol=1.00e-06  P=4
============================================
[Mode 1 - Dynamic]  P=4
  Result           : -0.258653611801624
  Accepted intervals: 3
  Time             : 0.000152 seconds
------------------------------------------------
  Timing summary (discarded run 0)
    run  0: 0.000317 s  [discarded]
    run  1: 0.000042 s
    run  2: 0.000043 s
    run  3: 0.000040 s
    run  4: 0.000006 s
    run  5: 0.000007 s
    run  6: 0.000005 s
    run  7: 0.000005 s
    run  8: 0.000005 s
    run  9: 0.000005 s
  Average (runs 1-9): 0.000018 seconds
================================================
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Assignment_3$ OMP_NUM_THREADS=1 mpirun -np 4  ./integration 2 2 1e-6
```